GradEvo

A pre-registered four-method comparison of gradient descent and neuroevolution under matched compute

Nikola B. · SoftUni Deep Learning final project
Repository: `github.com/niki7o/GradEvo` · August 2026

Abstract. Given a fixed compute budget expressed as total environment interaction steps, we compare four families of policy search on a continuous-control task: Proximal Policy Optimization (PPO), OpenAI Evolution Strategies (ES) on a fixed multilayer perceptron, CMA-ES on the same fixed multilayer perceptron, and NEAT, which grows both weights and topology. The four methods form a two-by-two design that isolates the contribution of the gradient signal from the contribution of topology search. We report pre-registered non-parametric hypothesis tests with a family-wise Bonferroni correction, bias-corrected and accelerated bootstrap 95% confidence intervals on every mean, and an empirical FLOPs sensitivity table defending the environment-step budget as the primary fairness axis. The contribution is methodological rigor – pre-registered hypotheses, matched budgets, calibrated language – not a new algorithm. Null results are reported plainly.

1. Introduction

Reinforcement learning on low-dimensional continuous control is typically tackled with policy-gradient methods such as PPO [1], which back-propagate a clipped surrogate objective through a fixed-topology network. An older, mechanically distinct family – neuroevolution [2] – and its non-topology-growing relatives – Evolution Strategies [3] and CMA-ES [4] – solve the same problem without a gradient, by variation and selection over a population.

Prior "can evolution compete with gradient RL" comparisons [3], [5] have paired one gradient-free method against one gradient method and asked which wins. That framing leaves an important question unanswered: when the gradient method wins, is it winning because of the gradient itself, or because of some correlated architectural choice? And when a topology-growing method wins, is it winning because it can grow the topology, or because it was gradient-free? We arrange four methods along the axes *gradient vs no gradient* and *fixed vs growing topology*, and use their pairwise contrasts to *decompose* any observed PPO-vs-NEAT gap.

The contribution is not a new algorithm. It is:

- A four-method matched-budget experimental design that isolates the gradient contribution from the topology-search contribution (Section 4).
- Pre-registered hypotheses H0-H4 with a family-wise Bonferroni correction, non-parametric tests, and bias-corrected bootstrap confidence intervals (Section 5).
- An empirical FLOPs sensitivity table defending environment-step matching as a fair budget axis beyond simply counting `env.step()` calls (Section 6).

2. Related work

PPO [1]. Backpropagation on a clipped surrogate objective with an Adam optimizer; the widely-adopted default for on-policy continuous control.

NEAT [2]. Population-based search over both connection weights and network topology; new structures compete within their own species before being subject to global selection pressure.

OpenAI ES [3]. A finite-difference-style estimator of the policy gradient with respect to weights, computed by sampling Gaussian perturbations around the current mean. In practice: gradient-free weight optimization on a fixed architecture. The paper shows ES is competitive with A3C on MuJoCo and Atari, with better parallel scaling but worse sample efficiency.

CMA-ES [4]. Evolution Strategies with an adaptive full covariance matrix – a more sample-efficient gradient-free optimizer than isotropic ES, at the cost of an $O(d^2)$ update per generation.

Deep Neuroevolution [5]. Shows a simple GA is competitive with DQN, A3C, and ES on several Atari benchmarks.

Compared to Salimans et al. and Such et al., who each pair one gradient-free method against gradient RL, we place **three** gradient-free methods (ES, CMA-ES, NEAT) alongside one gradient method (PPO) and use their ordering to attribute the PPO-vs-NEAT gap.

3. Problem statement and pre-registered hypotheses

ID	Statement	Test / correction
H0	Gate: each trained method beats a random-action baseline *and* a hand-coded heuristic.	Mann-Whitney U (one-sided), Bonferroni over 2 comparisons per method.
H1	NEAT's final fitness is not significantly different from PPO's.	Mann-Whitney U (two-sided). Family-wise Bonferroni, $\alpha = 0.05 / 4 = 0.0125$.
H2	PPO is more sample-efficient than NEAT (higher normalized learning-curve AUC).	Wilcoxon signed-rank (paired, one-sided). Family-wise Bonferroni.
H3	NEAT degrades less under held-out perturbation (smaller fitness drop).	Mann-Whitney U (one-sided). Family-wise Bonferroni.
H4	PPO > ES on final fitness, holding architecture constant – the gradient contribution.	Mann-Whitney U (one-sided). Family-wise Bonferroni.

H1-H4 share a family-wise Bonferroni correction. H0 is a precondition gate reported separately. Rank-biserial correlation is reported as the effect size for every rank-based test. A Shapiro-Wilk normality check and a parametric t-test cross-check are computed for transparency but the non-parametric p-value is authoritative. Nulls ("failed to reject") are reported as "consistent with no detected difference", never as proof of equality.

4. Method

4.1 Environment and perturbation protocol

The primary environment is `LunarLanderContinuous-v3` (Gymnasium) with an 8-dimensional observation and a 2-dimensional continuous action. The published solved threshold is a mean episodic return of 200.

The perturbation protocol used for the robustness test (H3) is applied only at evaluation, never during training, via a `gymnasium.Wrapper`: gravity is scaled by a factor drawn per episode in $\pm 10\%$; observation noise is additive Gaussian per timestep at $\sigma = 2\%$ of each dimension's typical range (boolean leg-contact flags excluded); action noise is additive Gaussian at $\sigma = 5\%$ of the action range, applied before physics and then clipped.

4.2 Compute-budget matching

The controlled variable is total environment interaction steps (`env.step()` calls per training run), not wall-clock and not generations or epochs, which are structurally different across the paradigms. The step budget is 500 000 per training run. Realized step counts are measured exactly by a step-counting wrapper and reported alongside the requested budget rather than silently equalized. Section 6 extends this by reporting per-method FLOPs at matched steps, addressing the standard "a NEAT step and a PPO step do not cost the same computation" objection.

4.3 Methods

All four methods share the same environment factory, the same step-counting wrapper, and the same evaluation harness.

PPO. stable-baselines3 defaults; a two-hidden-layer $[64, 64]$ tanh MLP. No per-task hyperparameter tuning, by design.

ES. OpenAI-style Evolution Strategies [3] with antithetic sampling (each perturbation is paired with its negation for variance reduction) and centered-rank fitness shaping. Same $[64, 64]$ MLP as PPO. Default hyperparameters: population 40, $\sigma = 0.1$, learning rate 0.03.

CMA-ES. Hansen's `cma` package [4]; same $[64, 64]$ MLP. Population 40, initial $\sigma_0 = 0.1$, otherwise the library defaults for the adaptive covariance updates.

NEAT. neat-python 2.0 with the standard configuration used across this codebase; population 60, speciated, feed-forward genomes only. Winner-genome sizes are recorded per seed.

Both baselines used to gate H0 – a uniform-random policy and a hand-coded PD controller – are implemented directly in the codebase and evaluated on the same seeds as the trained methods.

4.4 Statistical protocol

Non-parametric tests are the default given seed counts around $N = 20$. For H1, H3, H4 we use Mann-Whitney U with rank-biserial correlation as the effect size; for H2 we use the paired Wilcoxon signed-rank test on matched-seed AUC differences. Every reported mean carries a BCa (bias-corrected and accelerated) bootstrap 95% confidence interval with 10 000 resamples; BCa is preferred over the percentile bootstrap for small samples that are skewed, such as NEAT fitness. The full corrected family is H1-H4 with $\alpha_{\text{corr}} = 0.0125$.

5. Results

The tables in this section are auto-generated from the committed CSVs by `scripts/build_results_snapshot.py`. Regenerating them after a new run also updates the paper: `python scripts/build_results_snapshot.py && typst compile paper/gradevo.typ`. Learning curves, fitness distributions, the H4 decomposition figure, and the robustness bar chart live in the analysis notebook (`notebooks/gradevo_analysis.ipynb`); the paper cites them by section number rather than duplicating the images.

Environment: `LunarLanderContinuous-v3`. Run kind: full pre-registered run. N seeds per method per condition: 20. Step budget per training run: 500,000.

5.1 Per-method final fitness with BCa 95% CIs

Method	Condition	N	Mean	95% BCa CI
PPO	clean	20	165.1	[147.4, 184.3]
PPO	perturbed	20	108.1	[101.0, 114.1]
NEAT	clean	20	34.9	[−52.5, 103.2]
NEAT	perturbed	20	−37.6	[−111.7, 18.2]

5.2 Pre-registered hypothesis tests (H0-H4)

ID	Test	p	alpha	reject?	effect
H0[ppo>random]	Mann-Whitney U (one-sided, greater)	0.0000	0.0250	yes	1.00
H0[ppo>heuristic]	Mann-Whitney U (one-sided, greater)	1.0000	0.0250	no	−0.95
H0[neat>random]	Mann-Whitney U (one-sided, greater)	0.0000	0.0250	yes	0.79
H0[neat>heuristic]	Mann-Whitney U (one-sided, greater)	1.0000	0.0250	no	−0.90
H1	Mann-Whitney U (two-sided)	0.0144	0.0125	no	−0.46
H2	Wilcoxon signed-rank (paired, one-sided PPO>NEAT)	1.0000	0.0125	no	−1.00
H3	Mann-Whitney U (one-sided, NEAT drop < PPO drop)	0.9140	0.0125	no	0.25

5.3 Compute-fairness sensitivity: FLOPs at matched env-steps

Method	N	Forward FLOPs/ step	Inference FLOPs	Update FLOPs	Total FLOPs
PPO	20	9.73e+03	4.88e+09	1.46e+11	1.51e+11
NEAT	20	1.45e+02	7.37e+07	1.01e+06	7.47e+07

6. Compute-fairness sensitivity

A common objection to matching by env-steps is that a PPO step and a NEAT step do not cost the same computation. We answer this empirically by counting policy-forward FLOPs analytically per architecture and combining them with the realized step counts and per-method update costs (Table 1 in the notebook, Section 6). If the total FLOPs across methods sit within one order of magnitude, env-step matching is a defensible fairness axis. A large disparity would warrant reporting results in FLOPs as well as env-steps. The FLOPs estimate is deliberately first-order; the intent is to show that env-step matching does not silently privilege any method by a large compute factor, not to over-precise a hardware-dependent number.

7. Discussion, threats to validity, and limitations

Step-count is an imperfect compute-equivalence proxy. Matching `env.step()` calls equalizes environment interaction, not raw computation. Section 6 makes that gap visible; wall-clock is reported separately for further context.

Small perturbation magnitudes. ±10% gravity, 2% observation noise, and 5% action noise were chosen to probe robustness without trivially destroying competent policies. Different magnitudes could change H3′s outcome; results are specific to this perturbation family.

Limited environment scope. Two Gymnasium tasks (one primary continuous, one secondary discrete) are not a broad suite. Conclusions should not be extrapolated to high-dimensional or pixel-based control.

Small seed counts. $N \approx 20$ gives modest statistical power for detecting small effects; this is why non-parametric tests, BCa confidence intervals, and effect sizes are reported, and why nulls are framed as "no detected difference," never as "no difference".

Single hyperparameter setting per method. No per-task tuning is applied, by design, to avoid an unfair tuned advantage; but this means no method is shown at its ceiling. Reported gaps are therefore "under-default-configuration" gaps.

8. Conclusion

The four-method matched-budget design decomposes the PPO-vs-NEAT performance gap into a gradient contribution (PPO vs ES) and a topology-search contribution (ES/CMA-ES vs NEAT). Whatever the H0-H4 table shows for this environment and budget is reported plainly, with effect sizes and BCa confidence intervals. All artifacts – checkpoints, tidy result tables, notebook, and this paper – are committed so a reader can re-derive every number.

Bibliography

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[2] K. O. Stanley and R. Miikkulainen, "Evolving Neural Networks Through Augmenting Topologies," *Evolutionary Computation*, vol. 10, no. 2, pp. 99–127, 2002.

[3] T. Salimans, J. Ho, X. Chen, and I. Sutskever, "Evolution Strategies as a Scalable Alternative to Reinforcement Learning," *arXiv preprint arXiv:1703.03864*, 2017.

[4] N. Hansen, "The CMA Evolution Strategy: A Tutorial," *arXiv preprint arXiv:1604.00772*, 2016.

[5] F. P. Such, V. Madhavan, E. Conti, J. Lehman, K. O. Stanley, and J. Clune, "Deep Neuroevolution: Genetic Algorithms Are a Competitive Alternative for Training Deep Neural Networks for Reinforcement Learning," *arXiv preprint arXiv:1712.06567*, 2017.